

动态规划

背包问题

子集dp

数位dp

插头dp

斜率优化

四边形不等式

常见优化

动态规划

背包问题

01背包:

- 转移方程: $f[i][v] = \max(f[i-1][v], f[i-1][v-v_i] + w_i)$
- 时间复杂度: $O(nv)$

```
1  vector<int> dp(V + 1);
2  for (int i = 0; i < n; i++) {
3      auto dpn = dp;          //滚动一
4      for (int j = v[i]; j <= V; j++) {
5          dpn[j] = max(dp[j], dp[j - v[i]] + w[i]);
6      }
7      swap(dp, dpn);
8      //for (int j = V; j >= v[i]; j--) {      //滚动二
9      //    dp[j] = max(dp[j], dp[j - v[i]] + w[i]);
10     //}
11 }
```

完全背包:

- 转移方程: $f[i][v] = \max_{k=0}^{\lfloor \frac{v}{v_i} \rfloor} (f[i-1][v - k \times v_i] + k \times w_i)$
- 时间复杂度: $O(nv)$

```
1  vector<int> dp(V + 1);
2  for (int i = 0; i < n; i++) {      //完全背包无需滚动
3      for (int j = v[i]; j <= V; j++) {
4          dp[j] = max(dp[j], dp[j - v[i]] + w[i]);
5      }
6  }
```

多重背包:

- 转移方程: $f[i][v] = \max_{k=0}^{\min(k_i, \lfloor \frac{v}{v_i} \rfloor)} (f[i-1][v - k \times v_i] + k \times w_i)$
- 1. 时间复杂度(普通): $O(nvk)$

```

1  vector<int> dp(V + 1);
2  for (int i = 0; i < n; i++) {
3      while (K[i]--) {
4          auto dpn = dp;
5          for (int j = v[i]; j <= V; j++) {
6              dpn[j] = max(dp[j], dp[j - v[i]] + w[i]);
7          }
8          swap(dp, dpn);
9      }
10 }

```

2. 时间复杂度(二进制): $O(nv \log k)$

```

1  vector<int> dp(V + 1);
2  for (int i = 0; i < n; i++) {
3      for (int k = 1; k <= K[i]; K[i] -= k, k <= 1) { //进行二进制分组，每次使用后减掉
4          auto dpn = dp;
5          for (int j = v[i]; j <= V; j++) {
6              dpn[j] = max(dp[j], dp[j - k * v[i]] + k * w[i]);
7          }
8          swap(dp, dpn);
9      }
10 }

```

3. 时间复杂度(单调队列): $O(nr_i \lfloor \frac{v}{v_i} \rfloor) = O(nv)$

$$f[i][v] = \max_{k=0}^{\min(k_i, \lfloor \frac{v}{v_i} \rfloor)} (f[i-1][v - k \times v_i] + k \times w_i)$$

$$\text{令 } r = v \bmod v_i, \text{ 对 } \max \text{ 的 } k \text{ 进行换元成 } \begin{cases} v - k \times v_i & \rightarrow k \times v_i + r \\ k \times v_i & \rightarrow (\lfloor \frac{v}{v_i} \rfloor - k) \times w_i \\ v & \rightarrow \lfloor \frac{v}{v_i} \rfloor \times v_i + r \end{cases}$$

$$\Rightarrow f[i][\lfloor \frac{v}{v_i} \rfloor \times v_i + r] = \max_{k=0}^{\min(k_i, \lfloor \frac{v}{v_i} \rfloor)} (f[i-1][k \times v_i + r] + (\lfloor \frac{v}{v_i} \rfloor - k) \times w_i)$$

$$\Rightarrow f[i][\lfloor \frac{v}{v_i} \rfloor \times v_i + r] = \max_{k=0}^{\min(k_i, \lfloor \frac{v}{v_i} \rfloor)} (f[i-1][k \times v_i + r] - k \times w_i) + \lfloor \frac{v}{v_i} \rfloor \times w_i$$

```

1  vector<int> dp(V + 1);
2  pair<int, int> q[MAXK]; //滑动窗口
3
4  for (int i = 0; i < n; i++) {
5      auto dpn = dp;
6      K[i] = min(K[i], V / v[i]); //设置滑动窗口的长度
7
8      for (int r = 0; r < v[i]; r++) { //枚举余数
9          int head = 0, tial = -1; //设置活动窗口头和尾
10         for (int j = r, k = 0; j <= v; j += v[i], k++) {
11             while (head <= tial && q[head].second < k - K[i]) head++; //单调队列总长度大于设置的滑动窗口
12             //长度弹出
13             while (head <= tial && q[tial].first < dp[j] - k * w[i]) tial--; //维护单调性，头最大
14             q[++tial] = { dp[j] - k * w[i], k };
15             dpn[j] = q[head].first + k * w[i]; //此时的k恰好为 floor(v / v[i])
16         }
17     }
18     swap(dp, dpn);
19 }

```

树上背包:

- 思路: 如果子节点选了, 父亲也必需选

- 转移方程:

$$f[u][v] = \max_{son \text{ of } u} \{f[u][v], f[u][v - k - v_u] + f[son][k] + w_u\} \quad v \in [v_u, V], k \in [0, v - v_u]$$

1. 时间复杂度(普通): $O(nV^2)$

```

1  vector<vector<int>> dp(n + 1, vector<int>(V + 1));
2  vector<int> vis(n + 1);
3  function<void(int)> dfs = [&](int u) {
4      vis[u] = 1;
5      for (auto & son : adj[u]) {
6          if (vis[son]) continue;
7          dfs(son);
8          for (int j = v[u]; j <= V; j++) {
9              for (int k = 0; k <= j - v[u]; k++) {
10                 dp[u][j] = max(dp[u][j], dp[u][j - k - v[u]] + dp[son][k] + w[u]);
11             }
12         }
13     }
14 };

```

2. 时间复杂度(优化循环上下界): 均摊为 $O(nV)$

v 和 k 最多分别枚举至 $size(u)$ 和 $size(son)$

```

1  vector<vector<int>> dp(n + 1, vector<int>(V + 1));
2  vector<int> vis(n + 1), sz(n + 1);
3  auto dfs = [&](auto & dfs, int u) {
4      vis[u] = 1;
5      sz[u] = v[u];
6      dp[u][v[u]] = w[u];
7      for (auto & v : adj[u]) {
8          if (vis[v]) continue;
9          dfs(dfs, v);
10         sz[u] += sz[v];
11         auto dpn = vector<int>(sz[u] + 1);
12         for (int j = 0; j <= min(sz[u] - sz[v], V); j++) { //滚动
13             for (int k = 0; k <= min(sz[v], V - j); k++) {
14                 dpn[j + k] = max(dpn[j + k], dp[u][j] + dp[v][k]);
15             }
16         }
17         swap(dp[u], dpn);
18     }
19 };

```

子集dp

$$a \& b = 0 \Leftrightarrow a \subseteq \bar{b}, a \& b \neq 0 \Leftrightarrow a \subsetneq \bar{b}$$

使用高维前缀和求出当前集合(特定的)子集和 $f(S) = \sum_{T \setminus \text{subset } S} f(T)$

```

1  for (int i = 0; i < D; i++) {
2      for (int sta = 0; sta < (1ll << D); sta++) {
3          if (sta >> i & 1) {
4              f[sta] += f[sta ^ (1 << i)];
5          }
6      }
7  }

```

数位dp

作用: 解决限制大小问题

1. $[x, y] \rightarrow f(y) - f(x - 1)$, 超出 $long\ long$ 时考虑 $f(y) - f(x) + check(x)$
2. 从树的角度考虑

注: 是否包含0, 0需要考虑前导0

```

1  int a, b;

```

```

2  cin >> a >> b;
3
4  vector<pair<int, array<int, 10>>> dp(15, {-1, {}}); // dp 第二维记录每个数字出现次数
5  vector<int> ksm(15, 1);
6
7  for (int i = 1; i <= 14; i++) {
8      ksm[i] = ksm[i - 1] * 10;
9  }
10
11 auto getans = [&](int x) {
12     vector<int> num(1), sum(1);
13
14     for ( ; x; ) {
15         sum.push_back((x % 10) * ksm[num.size() - 1] + sum.back());
16         num.push_back(x % 10);
17         x /= 10;
18     }
19
20     auto dfs = [&](auto dfs, int pos, int lim, int f0) -> array<int, 10> {
21         if (!pos) {
22             return {0};
23         }
24         if (!lim && !f0 && (~dp[pos].first)) { // 无限制返回 dp
25             return dp[pos].second;
26         }
27
28         array<int, 10> res = {0};
29         int mx = lim ? num[pos] : 9;
30
31         for (int i = 0; i <= mx; i++) {
32             auto tmp = dfs(dfs, pos - 1, lim && i == num[pos], f0 && !i);
33             for (int j = 0; j <= 9; j++) {
34                 res[j] += tmp[j]; // 合并每个数字出现次数
35             }
36             if (f0 && !i) { // 当有前导零并且当前为 0 时
37
38             } else if (lim && i == num[pos]) { // 当当前已经最大时
39                 res[i] += sum[pos - 1] + 1;
40             } else { // 无限制情况
41                 res[i] += ksm[pos - 1];
42             }
43         }
44
45         if (!lim && !f0) { // 无限制情况更新 dp
46             dp[pos] = {0ll, res};
47         }
48
49         return res;
50     };
51
52     return dfs(dfs, num.size() - 1, 1, 1);
53 };
54
55 auto ta = getans(a - 1), tb = getans(b);
56
57 for (int i = 0; i <= 9; i++) {
58     cout << tb[i] - ta[i] << " ";
59 }

```

插头dp

```

1  int n, m;
2  cin >> n >> m;
3

```

```

4  vector<string> a(n + 1);
5
6  int end_i = 0, end_j = 0;
7
8  for (int i = 1; i <= n; i++) {
9      cin >> a[i];
10     a[i] = " " + a[i];
11     for (int j = 1; j <= m; j++) {
12         if (a[i][j] != '*') {
13             end_i = i, end_j = j;
14         }
15     }
16 }
17
18 unordered_map<int, int> dp;
19 dp[0] = 1;
20
21 for (int i = 1; i <= n; i++) {
22     auto get = [&](int state, int k) {
23         return state >> 2 * k & 3;
24     };
25     auto set = [&](int state, int k, int v) {
26         return state & ~(311 << 2 * k) | (v << 2 * k);
27     };
28     auto roll = [&]() {
29         unordered_map<int, int> dpn;
30         for (auto & [x, cnt] : dp) {
31             dpn[x << 2] += cnt;
32         }
33         swap(dpn, dp);
34     };
35
36     roll();
37
38     for (int j = 1; j <= m; j++) {
39         unordered_map<int, int> dpn;
40         for (auto & [mask, cnt] : dp) {
41             int lt = get(mask, j - 1), up = get(mask, j);
42             if (a[i][j] == '*') {
43                 if (!lt && !up) {
44                     dpn[mask] += cnt;
45                 }
46             } else {
47                 if (!lt && !up) {
48                     if (i + 1 <= n && a[i + 1][j] != '*' && j + 1 <= m && a[i][j + 1] != '*') {
49                         dpn[set(set(mask, j - 1, 1), j, 2)] += cnt;
50                     }
51                 } else if (!lt || !up) {
52                     int t = lt | up;
53                     if (i + 1 <= n && a[i + 1][j] != '*') {
54                         dpn[set(set(mask, j - 1, t), j, 0)] += cnt;
55                     } if (j + 1 <= m && a[i][j + 1] != '*') {
56                         dpn[set(set(mask, j - 1, 0), j, t)] += cnt;
57                     }
58                 } else {
59                     int state = set(set(mask, j - 1, 0), j, 0);
60                     if (lt == 1 && up == 1) {
61                         for (int u = j, s = 0; ; u++) {
62                             int z = get(mask, u);
63                             if (z == 1) {
64                                 s++;
65                             } else if (z == 2) {
66                                 if (--s == 0) {
67                                     dpn[set(state, u, 1)] += cnt;
68                                     break;
69                                 }

```

```

70         }
71     }
72     } else if (lt == 2 && up == 2) {
73         for (int u = j - 1, s = 0; ; u--) {
74             int z = get(mask, u);
75             if (z == 1) {
76                 if (++s == 0) {
77                     dpn[set(state, u, 2)] += cnt;
78                     break;
79                 }
80             } else if (z == 2) {
81                 s--;
82             }
83         }
84     } else if (lt == 2 && up == 1) {
85         dpn[state] += cnt;
86     } else if (lt == 1 && up == 2 && i == end_i && j == end_j) {
87         dpn[state] += cnt;
88     }
89 }
90 }
91 }
92 swap(dpn, dp);
93 }
94 }
95
96 cout << dp[0] << "\n";

```

斜率优化

考虑一次函数 $y = kx + b$ 转化为 $b = \min\{y - kx\}$ ，将与 j 有关的信息表示为 y ，把同时与 i, j 有关的信息表示为 kx ，把要最小化的信息（与 i 有关的信息）表示为 b （即等式左边常数部分）

查询找到一个 $K(q_{i-1}, q_i) \leq k < K(q_i, q_{i+1})$

- 斜率单调递增，新加的点横坐标也单调递增
 - 查询时可以将队头小于当前斜率的点全部删除
 - 在插入时，将队尾所有不在凸包上的点全部删除
- 斜率不单调，但新加的点横坐标单调递增
 - 查询时只能二分来找
 - 在插入的时候，将队尾所有不在凸包上的点全部删除
- 斜率单调，新加点横坐标不单调（同2）
- 均不单调使用，平衡树上二分或 CDQ 分治

```

1  deque<point> q;
2  q.push_back({0, 0});
3  vector<int> dp(n + 1);
4
5  //斜率单调递增，新加的点横坐标也单调递增
6  for (int i = 1; i <= n; i++) {
7      point tmp = {1, t[i] + s}; //随便设一条平行的向量
8      while (q.size() >= 2 && (q[1] - q[0]) * tmp >= 0) {
9          q.pop_front(); //弹出时考虑弹出斜率不优的
10     }
11     dp[i] = q[0][1] - tmp[1] * q[0][0] + s * c[n] + t[i] * c[i];
12     tmp = {c[i], dp[i]}; //新加入的点
13     while (q.size() >= 2 && (tmp - q[q.size() - 2]) * (q[q.size() - 1] - q[q.size() - 2]) >= 0) {
14         q.pop_back(); //加入时考虑维护凸包
15     }
16     q.push_back(tmp);
17 }
18

```

```

19 //斜率不单调，但新加的点横坐标单调递增
20 for (int i = 1; i <= n; i++) {
21     point tmp = {1, t[i] + s}; //随便设一条平行的向量
22
23     int l = 0, r = q.size() - 1;
24
25     auto chk = [&](int mid) { //二分查
26         return mid == 0 || (q[mid] - q[mid - 1]) * tmp >= 0;
27     };
28
29     while (l < r) {
30         int mid = l + r + 1 >> 1;
31         if (chk(mid)) {
32             l = mid;
33         } else {
34             r = mid - 1;
35         }
36     }
37
38     dp[i] = q[r][1] - tmp[1] * q[r][0] + s * c[n] + t[i] * c[i];
39     tmp = {c[i], dp[i]}; //新加入的点
40     while (q.size() >= 2 && (tmp - q[q.size() - 2]) * (q[q.size() - 1] - q[q.size() - 2]) >= 0) {
41         q.pop_back();
42     }
43     q.push_back(tmp);
44 }

```

四边形不等式

定义：

决策单调性： $\forall i_1 < i_2 \Rightarrow opt(i_1) \leq opt(i_2)$

四边形不等式： $\forall a \leq b \leq c \leq d \Rightarrow w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$

四边形恒等式： $\forall a \leq b \leq c \leq d \Rightarrow w(a, c) + w(b, d) = w(a, d) + w(b, c)$

区间包含单调性： $\forall a \leq b \leq c \leq d \Rightarrow w(b, c) \leq w(a, d)$

解决问题： 假设 $w(i, j)$ 可以 $O(1)$ 计算

1. **最优化问题：** $f(i) = \min_{1 \leq j \leq i} w(j, i)$
2. **区间拆分问题：** $f(i) = \min_{1 \leq j \leq i} f(j-1) + w(j, i)$
3. **限制区间个数的问题：** $f(k, i) = \min_{1 \leq j \leq i} f(k-1, j-1) + w(j, i)$
4. **区间合并问题：** $f(j, i) = \min_{j \leq k < i} f(j, k) + f(k+1, i) + w(j, i)$

定理：

1. $w(j, i)$ 满足四边形不等式 $\Rightarrow$ 问题 (1)，问题 (2)，问题 (3) 满足决策单调性，可使用二分队列，时间复杂度 $O(n \log n)$
2. $w(j, i)$ 满足四边形不等式且满足区间包含单调性 $\Rightarrow$ 问题 (4) 复杂度可优化至 $O(n^2)$

代码：

1. **二分队列：**

```

1 vector<int> dp;
2
3 auto F = [&](int n) {
4     dp = vector<int>(n + 1);
5     deque<array<int, 3>> q = {{0, 1, n}}; // 第一维为 i，第二维为 j，第三维为以 j 为决策点的最右端
6     for (int i = 1; i <= n; i++) {
7         auto w = [&](int j, int i) {
8             return w(j, i);
9         };

```

```

10     dp[i] = min(dp[i], w(q[0][0], i));
11     q[0][1] = i + 1;
12     if (q[0][1] > q[0][2]) {
13         q.pop_front();
14     }
15
16     auto insert = [&](int i) {
17         int pos = n + 1;
18         while (q.size() && w(q.back()[0], q.back()[1]) >= w(i, q.back()[1])) {
19             pos = q.back()[1];
20             q.pop_back();
21         }
22         if (q.size() && w(q.back()[0], q.back()[2]) >= w(i, q.back()[2])) {
23             auto [j, l, r] = q.back();
24             while (l < r) {
25                 int mid = l + r >> 1;
26                 if (w(j, mid) >= w(i, mid)) {
27                     r = mid;
28                 } else {
29                     l = mid + 1;
30                 }
31             }
32             q.back()[2] = r - 1;
33             pos = r;
34         }
35         if (pos <= n) {
36             q.push_back({i, pos, n});
37         }
38     };
39
40     insert(i);
41 }
42 };

```

2. 问题 (4) :

```

1  vector<vector<int>> dp, opt;
2  [&](int n) {
3      dp = vector(n + 1, vector<int>(n + 1, 4e18)), opt = vector(n + 1, vector<int>(n + 1));
4      for (int i = 1; i <= n; i++) { // 初始化
5          dp[i][i] = 0, opt[i][i] = i;
6      }
7      for (int len = 2; len <= n; len++) {
8          for (int j = 1, i = len; i <= n; j++, i++) {
9              for (int k = opt[j][i - 1]; k <= min(i - 1, opt[j + 1][i]); ++k) {
10                 if (dp[j][i] > dp[j][k] + dp[k + 1][i] + w(j, i)) {
11                     dp[j][i] = dp[j][k] + dp[k + 1][i] + w(j, i);
12                     opt[j][i] = k; // 更新最优决策点
13                 }
14             }
15         }
16     }
17 } ();

```

常见优化

1. 上下界优化:
2. 费用提前计算: 即动态局部费用转化为全局费用
3. 放宽限制 + 容斥原理: